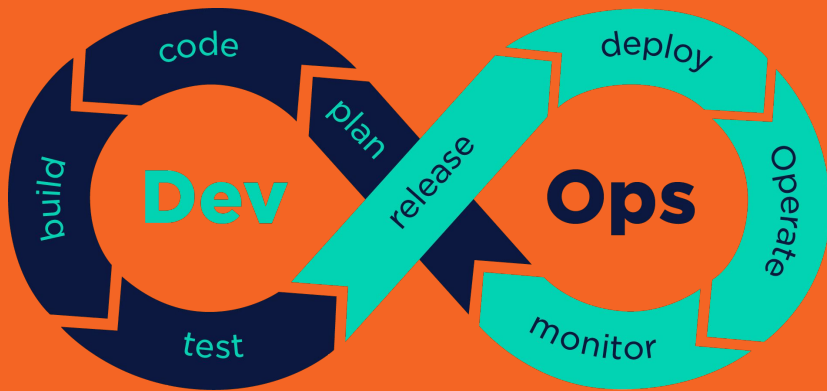

DevOps



Júlio Coutinho
DevOps Engineer Senior

Fundamentos e Cultura DevOps

DevOps é uma metodologia que visa unificar o desenvolvimento de software (Dev) e as operações de TI (Ops). Seu principal objetivo é encurtar o ciclo de vida de desenvolvimento de sistemas, proporcionando entrega contínua de alto valor com alta qualidade. Não se trata apenas de ferramentas, mas de uma mudança cultural que promove a colaboração, a comunicação e a integração entre as equipes de desenvolvimento e operações.



Cultura DevOps

A cultura DevOps é o pilar fundamental para o sucesso da implementação das práticas e ferramentas.

A colaboração e a comunicação são essenciais para que as equipes de desenvolvimento, operações, segurança e outras partes interessadas, promovam um ambiente de compartilhamento de conhecimento.

Além disso, a responsabilidade compartilhada garante que todos sejam responsáveis pelo sucesso do produto, desde a concepção até a operação em produção.

O Ciclo de Vida DevOps

A cultura DevOps é o pilar fundamental para o sucesso da implementação das práticas e ferramentas.

A colaboração e a comunicação são essenciais para que as equipes de desenvolvimento, operações, segurança e outras partes interessadas, promovam um ambiente de compartilhamento de conhecimento.

Além disso, a responsabilidade compartilhada garante que todos sejam responsáveis pelo sucesso do produto, desde a concepção até a operação em produção.

Fase	Descrição	Ferramentas Comuns (Exemplos)
Planejamento (Plan)	Definição de requisitos, priorização de funcionalidades e gestão ágil de projetos.	GitHub Projects, Jira
Desenvolvimento (Code)	Escrita de código, revisão por pares e controle de versão.	GitHub Repos, Git
Integração (Build)	Compilação do código, empacotamento e execução de testes unitários automatizados.	GitHub Actions, Jenkins
Teste (Test)	Execução de testes automatizados (integração, aceitação) e testes manuais exploratórios.	GitHub Actions, Selenium
Implantação (Release)	Preparação para implantação, gestão de aprovações e orquestração de ambientes.	GitHub Actions
Operação (Operate)	Gerenciamento da infraestrutura em produção e garantia de disponibilidade.	Prometheus, Grafana
Monitoramento (Monitor)	Coleta de métricas, logs e rastreamento de desempenho para feedback contínuo.	Prometheus, Grafana, ELK Stack

O Ciclo de Vida DevOps



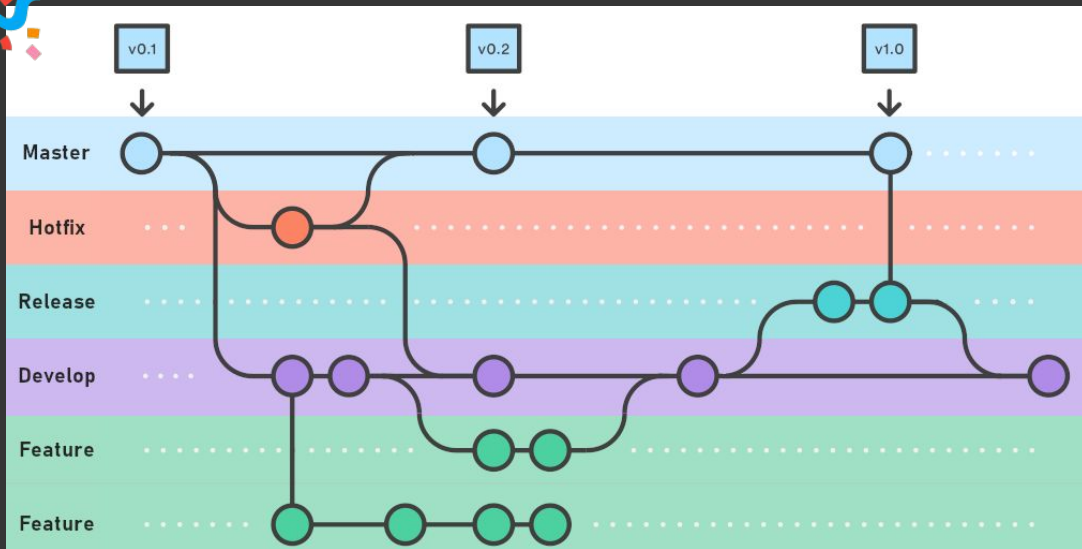
Métricas de Sucesso (DORA Metrics)

As métricas DORA (DevOps Research and Assessment) são um conjunto de quatro indicadores chave de desempenho que ajudam as equipes a medir a eficácia de suas práticas DevOps e a impulsionar a melhoria contínua.

Métricas de Sucesso (DORA Metrics)

Métrica DORA	O que mede	Objetivo Ideal
Deployment Frequency	Com que frequência a organização implanta código em produção com sucesso.	Sob demanda (múltiplas vezes ao dia).
Lead Time for Changes	O tempo que leva desde o commit do código até a execução bem-sucedida em produção.	Menos de uma hora.
Change Failure Rate	A porcentagem de implantações que resultam em degradação do serviço ou exigem correção.	Entre 0% e 15%.
Time to Restore Service	O tempo necessário para restaurar o serviço após uma interrupção ou falha em produção.	Menos de uma hora.

Controle de Versão e Estratégias de Branching



Git Essencial

O Git é um sistema de controle de versão distribuído, amplamente utilizado para rastrear mudanças em arquivos de código-fonte durante o desenvolvimento de software.

Comandos básicos: `git init`, `git clone`, `git pull`, `git push`, `git commit`, `git add` ...

Comandos Básicos do Git:

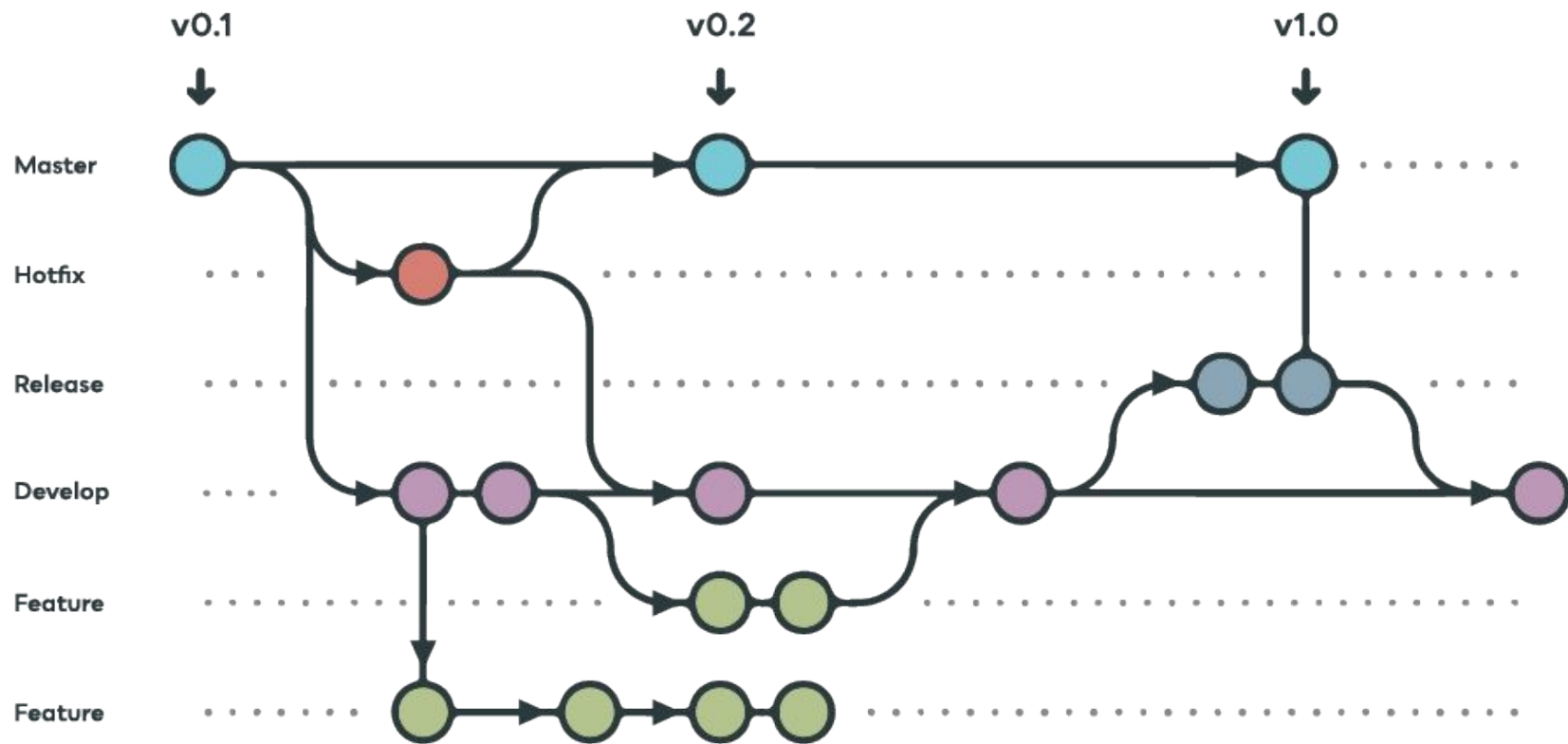
- `git init` : Inicializa um novo repositório Git no diretório atual.
- `git clone [URL]` : Cria uma cópia local de um repositório remoto.
- `git add .` : Adiciona todas as alterações no diretório de trabalho para a área de staging (preparação).
- `git commit -m "Mensagem do commit"` : Salva as alterações da área de staging no histórico do repositório local.
- `git status` : Mostra o estado atual do diretório de trabalho e da área de staging.
- `git push origin [branch]` : Envia os commits do repositório local para o repositório remoto (geralmente `origin` é o nome padrão do repositório remoto e `main` ou `master` é o branch principal).
- `git pull origin [branch]` : Baixa as alterações do repositório remoto e as mescla no branch local atual.
- `git branch` : Lista todos os branches locais.
- `git checkout [branch]` : Troca para um branch existente.
- `git checkout -b [novo-branch]` : Cria um novo branch e troca para ele.
- `git merge [branch-a-mesclar]` : Mescla as alterações de um branch em outro.

Estratégias de Branching

GitFlow: É um modelo de branching mais complexo e formal, ideal para projetos com ciclos de lançamento bem definidos e suporte a múltiplas versões.

Vantagens: Estrutura clara para projetos grandes e complexos, suporte a múltiplos lançamentos paralelos.

Desvantagens: Pode ser excessivamente complexo para equipes pequenas ou projetos com entrega contínua rápida.



Trunk-Based Development (TBD)

O Trunk-Based Development é uma estratégia mais simples e ágil, onde os desenvolvedores mesclam suas alterações em um único branch principal (o trunk ou main) de forma contínua e frequente.

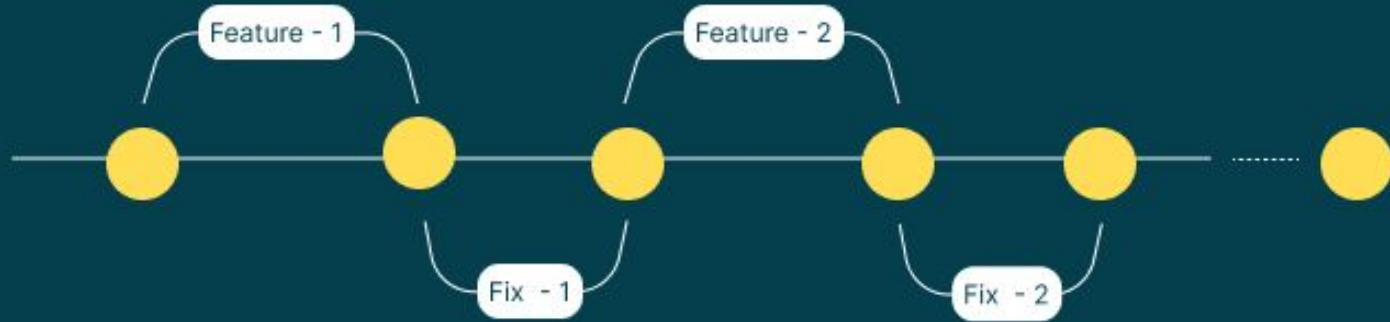
O objetivo é manter o branch principal sempre em um estado implantável, com pequenas e frequentes integrações.

Trunk-Based Development (TBD)

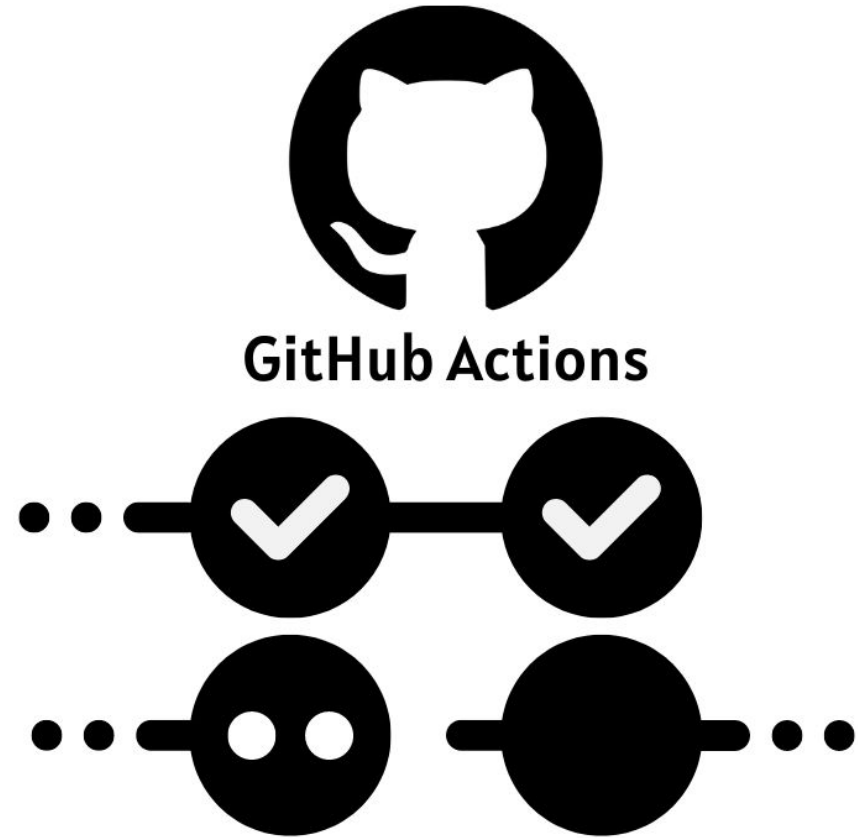
Vantagens: Promove a Integração Contínua verdadeira, reduz a complexidade de merges, acelera o feedback e a entrega. Ideal para equipes que praticam entrega contínua e implantação contínua.

Desvantagens: Requer alta disciplina de testes automatizados e commits pequenos e frequentes para evitar quebras no branch principal.

Trunk Based Development



CI/CD com GitHub Actions



Conceitos de CI/CD

Integração Contínua (CI - Continuous Integration) é uma prática de desenvolvimento de software onde os desenvolvedores integram seu código em um repositório compartilhado várias vezes ao dia.

Cada integração é verificada por um build automatizado (incluindo testes automatizados) para detectar erros de integração o mais cedo possível. O objetivo principal da CI é garantir que o código base esteja sempre em um estado funcional e integrável.

Conceitos de CI/CD

Entrega Contínua (CD - Continuous Delivery) é uma extensão da Integração Contínua, onde o software é construído de tal forma que pode ser liberado para produção a qualquer momento.

Isso significa que, após a fase de CI, o código é automaticamente preparado para implantação, passando por estágios adicionais de teste e validação em ambientes que simulam a produção. A decisão de implantar em produção ainda é manual.

Conceitos de CI/CD

Implantação Contínua (CD - Continuous Deployment) vai um passo além da Entrega Contínua. Com a Implantação Contínua, cada alteração que passa por todos os estágios do pipeline de produção é automaticamente liberada para os usuários finais, sem intervenção humana.

Isso requer um alto nível de automação e confiança nos testes e no pipeline.

GitHub Actions

GitHub Actions é uma plataforma de CI/CD que permite automatizar tarefas de desenvolvimento de software diretamente no seu repositório GitHub.

Você pode criar workflows personalizados que são acionados por eventos no seu repositório (como pushes, pull requests, ou agendamentos) para construir, testar e implantar seu código.

Componentes Principais do GitHub Actions:

Workflow: Um processo automatizado configurável que é executado em um ou mais jobs. Workflows são definidos em arquivos YAML (.yml) localizados no diretório `.github/workflows/` do seu repositório.

Eventos: Atividades específicas que acionam um workflow. Exemplos incluem `push`, `pull_request`, `schedule`, `workflow_dispatch` (acionamento manual).

Componentes Principais do GitHub Actions:

Jobs: Um conjunto de passos que são executados em um runner. Por padrão, jobs são executados em paralelo, mas você pode definir dependências entre eles.

Steps: Uma sequência de tarefas dentro de um job. Cada step pode ser um script de shell ou uma action.

Actions: Unidades de código reutilizáveis que executam tarefas específicas, como configurar um ambiente, fazer login em um provedor de nuvem, ou publicar um pacote. Você pode usar actions criadas pela comunidade, pelo GitHub, ou criar as suas próprias.

Componentes Principais do GitHub Actions:

Runners: Servidores que executam seus workflows. O GitHub fornece runners hospedados (Ubuntu, Windows, macOS), mas você também pode configurar seus próprios runners auto-hospedados.

Demonstração Prática de Automação com GitHub Actions

Cenário:

- Um aplicativo web .NET Core simples que exibe uma mensagem "Hello, GitHub Actions!".

O que vamos fazer:

- Configurar um workflow de CI para construir o aplicativo e executar testes unitários.
- Configurar um workflow de CD para implantar o aplicativo em um Azure App Service.

Demonstração Prática de Automação com GitHub Actions

Pré-requisitos:

- Conta GitHub e repositório com um aplicativo .NET Core ou Python.
- (Opcional para CD) Conta Azure e assinatura ativa, com um Azure App Service criado.

Infraestrutura como Código (IaC) e Monitoramento



Infraestrutura como Código (IaC) e Monitoramento

Infraestrutura como Código (IaC) é a prática de gerenciar e provisionar infraestrutura de TI usando arquivos de configuração legíveis por máquina;

Os principais benefícios do IaC incluem:

- Consistência: Garante que os ambientes sejam idênticos.
- Repetibilidade: Permite recriar ambientes de forma rápida e confiável.
- Velocidade: Acelera o provisionamento de infraestrutura.
- Redução de Erros: Minimiza erros humanos associados à configuração manual.
- Documentação: Os arquivos de configuração servem como documentação viva da infraestrutura.
- Parametrização: Permite modificar ambientes da forma desejada.

Containerização: Docker

Containerização é uma tecnologia que empacota um aplicativo e todas as suas dependências (bibliotecas, frameworks, configurações) em um "contêiner" isolado.

- Docker: É a plataforma de containerização mais popular. Ele permite criar, implantar e executar aplicativos em contêineres. Um Dockerfile define como construir uma imagem de contêiner, que é então usada para criar instâncias de contêiner.
- Kubernetes: Um orquestrador de contêineres de código aberto que automatiza a implantação, o dimensionamento e o gerenciamento de aplicativos em contêineres. Ele agrupa contêineres em unidades lógicas para fácil gerenciamento e descoberta. No contexto de DevOps, Docker e Kubernetes são fundamentais para criar ambientes consistentes, escaláveis e portáteis, facilitando a entrega contínua e a operação de microsserviços.

Monitoramento e Feedback

O monitoramento é uma parte crítica do ciclo de vida DevOps, fornecendo visibilidade sobre o desempenho e a saúde dos aplicativos e da infraestrutura em produção.

O feedback contínuo obtido através do monitoramento permite que as equipes identifiquem e resolvam problemas rapidamente, além de informar futuras melhorias.

Monitoramento e Feedback

Ferramentas de monitoramento populares incluem:

- Prometheus: Um sistema de monitoramento e alerta de código aberto que coleta métricas de alvos configurados em intervalos definidos, avalia expressões de regras, exibe os resultados e pode acionar alertas.
- Grafana: Uma plataforma de código aberto para análise e visualização de dados. Ele permite criar dashboards interativos e personalizados a partir de diversas fontes de dados, incluindo o Prometheus.

Segurança em DevOps (DevSecOps)

DevSecOps integra práticas de segurança em todas as fases do ciclo de vida DevOps. O objetivo é "shift left" (deslocar para a esquerda) a segurança, ou seja, introduzir verificações e considerações de segurança o mais cedo possível no processo de desenvolvimento, em vez de tratá-las como um pensamento posterior.

Segurança em DevOps (DevSecOps)

Principais recursos:

- Code scanning: Analisa o código-fonte em busca de vulnerabilidades de segurança e erros de codificação. Ele usa o CodeQL para encontrar problemas em seu código.
- Dependabot: Ajuda a manter suas dependências atualizadas e seguras, alertando sobre vulnerabilidades conhecidas em bibliotecas e frameworks que seu projeto utiliza e criando pull requests para atualizá-las.
- Secret scanning: Escaneia seu repositório em busca de segredos (tokens, chaves de API, credenciais) que foram acidentalmente expostos, alertando você para revogá-los e protegê-los.

Encerramento e Próximos Passos

DevOps é um campo em constante evolução. Para continuar crescendo, considere os seguintes tópicos:

- Cloud Computing Avançado: Aprofunde-se em serviços específicos de provedores de nuvem (Azure, AWS, GCP) como Functions, Kubernetes Service (AKS, EKS, GKE), Service Bus, etc.
- Observabilidade: Explore ferramentas e práticas para monitoramento mais avançado, tracing distribuído (OpenTelemetry, Jaeger) e análise de logs (ELK Stack, Grafana Loki).
- Segurança (DevSecOps): Estude ferramentas de segurança de pipeline, gerenciamento de identidade e acesso (IAM), e conformidade.

Encerramento e Próximos Passos

- Automação Avançada: Explore automação de infraestrutura com ferramentas como Ansible, Chef, Puppet, e aprofunde-se em Terraform/Bicep.
- Engenharia de Confiabilidade do Site (SRE): Entenda os princípios de SRE, SLIs, SLOs e SLAS, e como aplicá-los para construir sistemas mais confiáveis.
- FinOps: Aprenda a otimizar os custos da nuvem e a cultura de responsabilidade financeira em DevOps.

Encerramento e Próximos Passos

- Automação Avançada: Explore automação de infraestrutura com ferramentas como Ansible, Chef, Puppet, e aprofunde-se em Terraform/Bicep.
- Engenharia de Confiabilidade do Site (SRE): Entenda os princípios de SRE, SLIs, SLOs e SLAS, e como aplicá-los para construir sistemas mais confiáveis.
- FinOps: Aprenda a otimizar os custos da nuvem e a cultura de responsabilidade financeira em DevOps.

Q&A



Obrigado, Júlio
